

SEG3502 – Lab 5

Angular

Persistence et Authentification avec Firebase

L'objectif de cet laboratoire est d'examiner la persistance et l'authentification d'applications Angular à l'aide d'un service basé sur le cloud. Nous allons étendre l'application *Address Book* du Lab 3, en enregistrant et en lisant les entrées d'adresses à partir d'une base de données, et ayant des comptes de carnet d'adresses privés.

Le code source du laboratoire est disponible dans le référentiel Github

<https://github.com/stephanesome/address-book-firebase.git>

Persistence des entrées d'adresse

L'application *Address Book* ne stocke les entrées d'adresses qu'en mémoire. Ce qui n'est pas très utile. Nous allons étendre l'application avec la capacité de rendre les entrées d'adresses persistantes. Nous pourrions enregistrer les données localement au niveau client, mais cela ne supporterait pas plusieurs usagers ou ne permettrait pas l'utilisation de clients alternatifs tels que les clients mobiles.

Google Firestore

Nous utiliserons la base de données *Google Firebase Firestore*

(<https://firebase.google.com/products/firestore>). Une base de données NoSQL en temps réel qui permet de synchroniser les applications avec le magasin de données. Les données sont stockées sous forme de Collections constituées de Documents.

Le module AngularFire (<https://github.com/angular/angularfire>) doit être ajouté au projet pour pouvoir interagir avec Firestore. Dans le dossier racine du projet, exécutez la commande

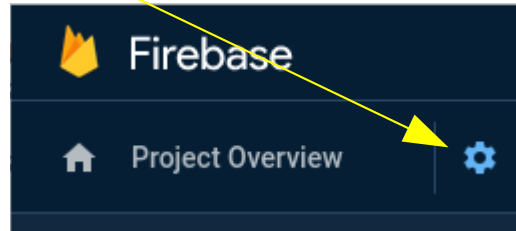
`ng add @angular/fire` pour ajouter AngularFire au projet. Sélectionnez pour configurer la fonctionnalité *Firestore*.

Configuration de Firestore

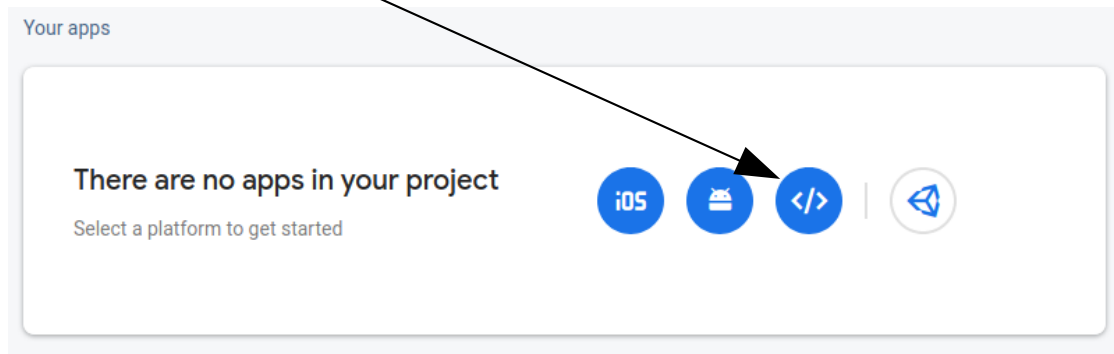
Les instructions de configuration d'une base de données Firestore Cloud sont décrites à l'adresse <https://firebase.google.com/docs/firestore/quickstart>. Nous allons résumer ici.

- Connectez-vous à la console Firebase <https://console.firebase.google.com/u/0/> puis ajoutez un projet (vous devez avoir un compte Google).
- Cliquez sur *Firestore Database* puis sélectionnez *Create database*. Sélectionnez de débiter en mode test (*Start in test mode*).
- Sélectionnez une location puis *Enable*.

- Sélectionnez les Settings du projet



- Ajoutez une application Web au projet.



- Générez des fichiers de configuration d'environnement en exécutant la commande `ng g environments`
- Ouvrez le fichier `src/environments/environment.ts` et copiez les données de configuration de Firestore Web App vers l'objet `firebase` de l'environnement de l'application. La valeur modifiée doit être similaire à ce qui suit avec les valeurs correspondant à vos paramètres.

```

1. export const environment = {
2.   apiKey: '<your-key>',
3.   authDomain: '<your-project-authdomain>',
4.   databaseURL: '<your-database-URL>',
5.   projectId: '<your-project-id>',
6.   storageBucket: '<your-storage-bucket>',
7.   messagingSenderId: '<your-messaging-sender-id>',
8.   appId: '<your-app-id>',
9.   measurementId: '<your-measurement-id>'
10. }
11. };

```

- Configurez l'instance du fournisseur Firestore dans `src/app.config.ts`.

```

1. import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
2. import { provideRouter } from '@angular/router';
3.
4. import { routes } from './app.routes';

```

```

5. import {initializeApp, provideFirebaseApp} from "@angular/fire/app";
6. import {getFirestore, provideFirestore} from "@angular/fire/firestore";
7. import {environment} from "../environments/environment.development";
8.
9. export const appConfig: ApplicationConfig = {
10.   providers: [
11.     provideZoneChangeDetection({ eventCoalescing: true }),
12.     provideRouter(routes),
13.     provideFirebaseApp(() => initializeApp(environment.firebase)),
14.     provideFirestore(() => getFirestore())
15.   ]
16. };

```

L'appel de la fonction `initializeApp()` est nécessaire pour transmettre l'objet de configuration ajouté précédemment dans le fichier `src/environments/environment.ts`.

Entrées d'Adresses

Nous ajouterons un champ `id` à la classe de modèle pour `AddressEntry`. Le champ servira d'identifiant unique aux entrées d'adresse stockées.

Modifiez la classe (`src/app/address-list/address-entry.ts`) comme suit

```

1. export class AddressEntry {
2.   public id?: string | null;
3.   public firstName: string;
4.   public lastName: string;
5.   public phone?: string;
6.   public email?: string;
7.   public notes?: string;
8.
9.
10.  constructor(firstName: string, lastName: string, phone?: string, email?: string, notes?: string) {
11.    this.id = null;
12.    this.firstName = firstName;
13.    this.lastName = lastName;
14.    this.phone = phone;
15.    this.email = email;
16.    this.notes = notes;
17.  }
18. }

```

Nous avons ajouté un champ `id` à la classe (ligne 2) et l'initialisons à `null` dans le constructeur (ligne 11).

Option Save

Nous ajoutons un bouton à `Address View` qu'un utilisateur peut utiliser pour sauvegarder une entrée d'adresse.

Template HTML d'Address View

Modifier le template HTML du composant Address View (src/app/address-view/address-view.html)

```
1. <form class="ui large form segment" #addressForm="ngForm">
2.   <h3 class="ui header">Address Details</h3>
3.   <fieldset [disabled]="!edit ? 'disabled' : null">
4.     <div class="form-group">
5.       <label for="firstName">First Name</label>
6.       <input class="form-control" name="firstName" id="firstName" [(ngModel)]="address.firstName">
7.     </div>
8.     <div class="form-group">
9.       <label for="lastName">Last Name</label>
10.      <input class="form-control" name="lastName" id="lastName" [(ngModel)]="address.lastName">
11.    </div>
12.    <div class="form-group">
13.      <label for="phone">Phone Number</label>
14.      <input class="form-control" name="phone" id="phone" [(ngModel)]="address.phone">
15.    </div>
16.    <div class="form-group">
17.      <label for="email">Email</label>
18.      <input class="form-control" name="email" id="email" [(ngModel)]="address.email">
19.    </div>
20.    <div class="form-group">
21.      <label for="notes">Notes</label>
22.      <textarea class="form-control" rows="4" cols="60" name="notes" id="notes"
    [(ngModel)]="address.notes"></textarea>
23.    </div>
24.  </fieldset>
25.
26.  <div class="btn-toolbar" role="toolbar">
27.    <button class="btn btn-danger" name="delete" (click)="delete()">Delete</button>
28.
29.    @if (edit) {
30.      <button (click)="toggleEdit()" class="btn btn-secondary m-auto">
31.        Protect
32.      </button>
33.    } @else {
34.      <button (click)="toggleEdit()" class="btn btn-secondary m-auto">
35.        Edit
36.      </button>
37.    }
38.    @if (!addressForm.pristine) {
39.      <button class="btn btn-dark" name="save" (click)="save()">Save</button>
40.    }
41.  </div>
42. </form>
```

Nous avons ajouté un bouton **Save** (lignes 38-40). La propriété **hidden** est définie de manière à ce que le bouton ne s'affiche que si le formulaire a été modifié. Une référence au **ngForm** (ajoutée à la ligne 1) permet d'accéder au statut du formulaire.

Class Address View

Le gestionnaire de l'événement clic du bouton **Save** est implémenté dans la classe de composant comme suit.

```
1. import {Component, EventEmitter, Input, OnInit, Output} from '@angular/core';
2. import {AddressEntry} from '../address-entry';
3.
4. @Component({
5.   selector: 'app-address-view',
6.   templateUrl: './address-view.html',
7.   styleUrls: ['./address-view.css']
8. })
9. export class AddressView implements OnInit {
10.   @Input() address: AddressEntry;
11.   @Output() fireDelete: EventEmitter<AddressEntry> = new EventEmitter();
12.   @Output() fireSave: EventEmitter<AddressEntry> = new EventEmitter();
13.   public edit: boolean;
14.
15.   constructor() { }
16.
17.   ngOnInit(): void {
18.     this.edit = true;
19.   }
20.
21.   toggleEdit(): void {
22.     this.edit = !this.edit;
23.   }
24.
25.   delete(): void {
26.     this.fireDelete.emit(this.address);
27.   }
28.
29.   save(): void {
30.     this.fireSave.emit(this.address);
31.   }
32. }
```

La fonction **save** (lignes 29 à 31) émet un événement output (ligne 12) notifiant le composant parent de la demande de suppression.

Service Firestore

Nous créons une classe de service pour encapsuler l'interaction avec la base de données Firestore. Le service serait injecté dans d'autres parties de l'application qui ont besoin d'un tel accès.

Générer un service `ng generate service address-list/firestore/address-db-service`. Modifiez `src/app/address-list/firestore/address-db-service.ts` comme suit.

```
1. import { Injectable, inject } from '@angular/core';
2. import {
3.   Firestore,
4.   collection,
5.   collectionData,
6.   addDoc,
7.   doc,
8.   setDoc,
9.   deleteDoc
10. } from '@angular/fire/firestore';
11. import { Observable } from "rxjs";
12. import { AddressEntry } from "../address-entry";
13.
14. @Injectable({
15.   providedIn: 'root'
16. })
17. export class AddressDbService {
18.   private firestore: Firestore = inject(Firestore);
19.
20.   getAddresses(): Observable<AddressEntry[]> {
21.     const addresses = collection(this.firestore, 'abooks', 'user', 'addresses');
22.     return collectionData(addresses, {idField: 'id'}) as Observable<AddressEntry[]>;
23.   }
24.
25.   createAddress(address: AddressEntry) {
26.     const addresses = collection(this.firestore, 'abooks', 'user', 'addresses');
27.     delete address.id;
28.     // @ts-ignore
29.     return addDoc(addresses, address);
30.   }
31.
32.   updateAddress(address: AddressEntry) {
33.     const addressId = address.id;
34.     delete address.id;
35.     const addresses = collection(this.firestore, 'abooks', 'user', 'addresses');
36.     const addressRef = doc(addresses, addressId!);
37.     // @ts-ignore
38.     return setDoc(addressRef, address);
39.   }
40.
41.   deleteAddress(addressId: string): Promise<void> {
42.     const addresses = collection(this.firestore, 'abooks', 'user', 'addresses');
43.     const addressRef = doc(addresses, addressId);
44.     return deleteDoc(addressRef);
45.   }
46. }
```

Le module Angular Firestore fournit une API pour interagir avec la base de données Firestore. L'API utilise la communication asynchrone implémentée avec des **observables** et des **promises**. Une référence au service de magasin de données est fournie par une instance de **Firestore** injectée à la ligne 18.

La fonction `getAddresses` (lignes 20-23) retourne un Observable de documents (instances de `AddressEntry`). Nous commençons par appeler la fonction `collection` qui retourne une référence à une collection de documents basée sur un chemin. Les documents sont récupérés ou ajoutés à une collection à l'aide de cette référence. La fonction `collectionData` récupère et retourne les données de manière asynchrone dans un observable (ligne 22).

La fonction `createAddress` ajoute un document à la collection. Notez que Firestore crée automatiquement une collection lorsqu'elle n'existe pas. `updateAddress` modifie un document existant et `deleteAddress` supprime un document.

Composant Address List

Nous modifions à présent le composant **Address List** pour obtenir les entrées d'adresses à partir du service `AddressDbService` et pour gérer la sauvegarde ainsi que la suppression des entrées.

Composant Address List Template HTML

Modifiez le Template HTML (`src/app/address-list/address-list.html`) comme suit.

```
1. <div class="row">
2.   <div class="col-md-12">
3.     <button class="btn btn-success" (click)="addAddress()">New Address</button>
4.   </div>
5.   <div class="col-md-8" [ngStyle]="msgStyle" [hidden]="hideMsg">
6.     {{message}}
7.   </div>
8. </div>
9. <hr>
10. <div class="row">
11.   <div class="address-list col-md-3">
12.     @for (address of addresses; track $index) {
13.       <app-address-list-element
14.         [address]="address"
15.         (click)="select(address)"
16.       >
17.     </app-address-list-element>
18.   }
19. </div>
20.   <div class="col-md-9">
21.     @if (currentAddress !== null) {
22.       <app-address-view
23.         [address]="currentAddress"
24.         (fireDelete)="deleteCurrent()"
25.         (fireSave)="saveCurrent()"></app-address-view>
```

```
26.   }
27. </div>
28. </div>
```

Nous avons ajouté un gestionnaire pour l'événement `fireSave` qui est émis par le composant `Address View` lorsqu'un utilisateur clique pour enregistrer une entrée (ligne 25).

Un message s'affiche (lignes 5 à 7) pour fournir une rétroaction à l'utilisateur. Le style est contrôlé par une directive `ngStyle` et le message est masqué ou affiché en fonction de la liaison de propriété à une variable dans la classe de composant (`hideMsg`).

Class de composant d'Address List

La classe de composant de `Address List` est modifiée pour obtenir les entrées d'adresse à partir du service `AddressDb`.

```
1. import {Component, inject, OnInit} from '@angular/core';
2. import {AddressEntry} from './address-entry';
3. import {NotificationService} from './notification-service';
4. import {AddressListElement} from './address-list-element/address-list-element';
5. import {AddressView} from './address-view/address-view';
6. import {NgForOf, NgIf, NgStyle} from '@angular/common';
7. import {AddressDbService} from './firebase/address-db-service';
8.
9. @Component({
10.   selector: 'app-address-list',
11.   standalone: true,
12.   imports: [
13.     AddressListElement,
14.     AddressView,
15.     NgForOf,
16.     NgIf,
17.     NgStyle
18.   ],
19.   templateUrl: './address-list.html',
20.   styleUrls: ['./address-list.css'],
21.   providers: [NotificationService]
22. })
23. export class AddressList implements OnInit {
24.   addresses: AddressEntry[] = [];
25.   currentAddress: AddressEntry | null = null;
26.   message: string = '';
27.   hideMsg = true;
28.   msgStyle = {
29.     color: '',
30.     'background-color': 'white',
31.     'font-size': '150%',
32.   };
33.   private notificationService: NotificationService = inject(NotificationService);
34.   private store: AddressDbService = inject(AddressDbService);
```

```

35.
36. ngOnInit(): void {
37.   this.message = "";
38.   this.store.getAddresses().subscribe(data => {
39.     this.addresses = data.map(e => {
40.       return {
41.         id: e.id,
42.         ...e
43.       } as AddressEntry;
44.     });
45.   });
46. }
47.
48. select(address: AddressEntry): void {
49.   this.currentAddress = address;
50.   this.notificationService.selectionChanged(address);
51. }
52.
53. showMessage(type: string, msg: string): void {
54.   this.msgStyle.color = type === 'error' ? 'red' : 'blue';
55.   this.message = msg;
56.   this.hideMsg = false;
57.   setTimeout(
58.     () => {
59.       this.hideMsg = true;
60.     }, 2500
61.   );
62. }
63.
64. addAddress(): void {
65.   const newAddress = new AddressEntry('New', 'Entry');
66.   this.addresses = [newAddress, ...this.addresses];
67.   this.select(newAddress);
68. }
69.
70. deleteCurrent(): void {
71.   if (this.currentAddress && this.currentAddress.id !== null) {
72.     this.addresses =
73.       this.addresses.filter((address: AddressEntry) => address !== this.currentAddress);
74.     // **** permanently delete
75.     this.store.deleteAddress(this.currentAddress.id!)
76.       .then(_ =>
77.         this.showMessage('info', 'The address entry was successfully deleted')
78.       )
79.       .catch(_ =>
80.         this.showMessage('error', 'Error unable to delete the address entry')
81.       );
82.     this.currentAddress = null;
83.   }

```

```

84. }
85.
86. saveCurrent(): void {
87.   if (this.currentAddress && this.currentAddress.id === null) {
88.     this.store.createAddress(this.currentAddress)
89.       .then(
90.         (docRef: any) => {
91.           this.currentAddress!.id = docRef.id;
92.           this.showMessage('info', 'The address entry was successfully saved');
93.         }
94.       )
95.       .catch(_: any) =>
96.         this.showMessage('error', 'Error unable to save the address entry')
97.       );
98.   } else {
99.     this.store.updateAddress(this.currentAddress!)
100.       .then(_: any) =>
101.         this.showMessage('info', 'The address entry was successfully updated')
102.       )
103.       .catch(_: any) =>
104.         this.showMessage('error', 'Error unable to update the address entry')
105.       );
106.   }
107. }
108. }

```

Nous utilisons le service `AddressDb` injecté dans le constructeur (ligne 34). Le hook de cycle de vie `ngOnInit` (lignes 36-46) récupère tous les documents et initialise le tableau des adresses avec.

La fonction `deleteCurrent` (lignes 70-84) a été modifiée pour invoquer le service `AddressDb` pour la suppression réelle. L'appel à `deleteAddress` (ligne 75) retourne une `Promise`. La fonction `then` spécifie un gestionnaire pour le cas d'une notification de réussite tandis que `catch` prend en compte les situations d'erreur. Nous avons défini les messages en fonction du statut retourné (ligne 77 et ligne 80).

Une approche similaire est utilisée pour la fonction `saveCurrent` avec l'ajout qu'elle détermine si une nouvelle entrée est en cours de création ou qu'une mise à jour est effectuée en vérifiant la valeur de l'*id* de l'entrée d'adresse. Lorsqu'une nouvelle entrée est enregistrée, nous récupérons l'identifiant du document généré par Firestore et le définissons comme *id* pour la nouvelle entrée d'adresse (lignes 91-92).

Authentification avec Firebase

Nous allons à présent mettre à jour l'application avec l'authentification Firebase (<https://firebase.google.com/docs/auth>). Cela permettra à notre application d'être utilisée par plusieurs usagers. Chaque usager disposera d'un compte et ne pourra accéder qu'aux adresses qu'elle a enregistrées.

Nous allons ajouter une page d'inscription (Sign-Up)

Address Book

Sign Up

Email:

Name:

Password:

Confirm Password:

Register

Or

Continue with Google

Already have an account? [Log In](#)

Une page de connexion (Sign-In)

Address Book

Sign In

Email

Password

Log in

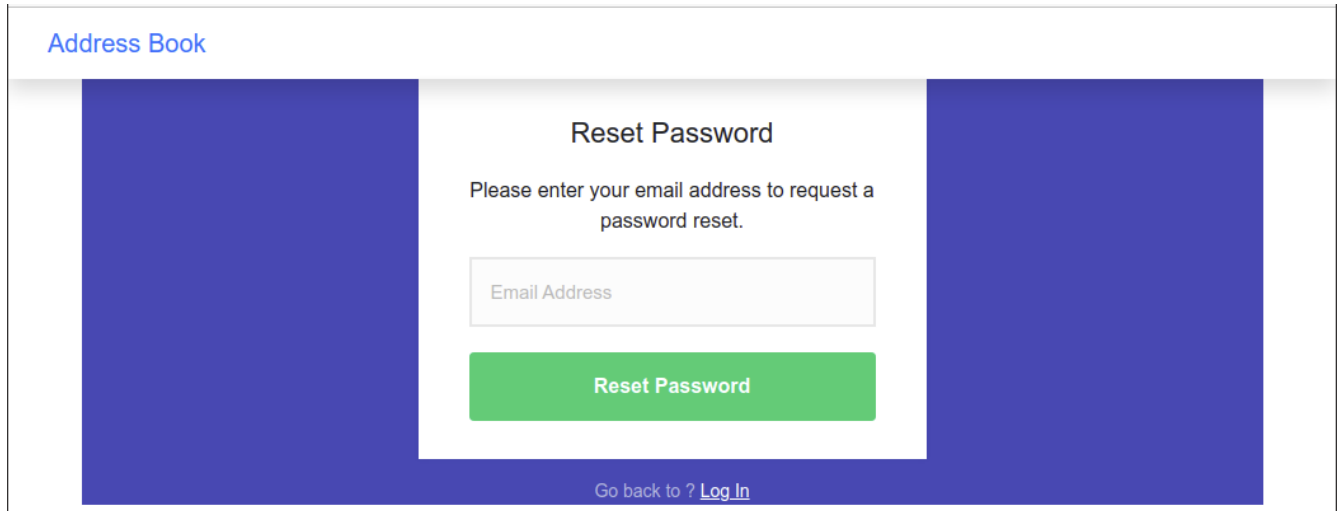
Or

Log in with Google

[Forgot Password?](#)

Don't have an account? [Sign Up](#)

Une page de réinitialisation de mot-de-passe (Password-Forgot)

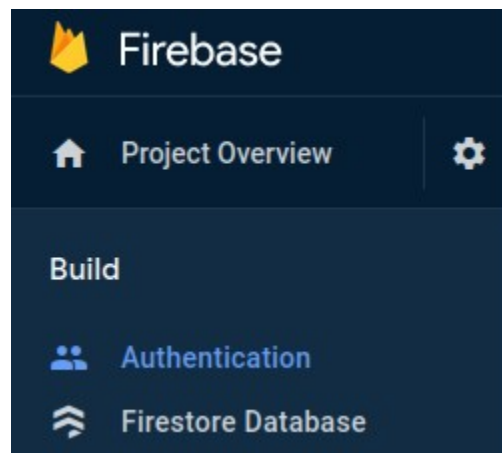


The screenshot shows a web page titled "Address Book" in the top left corner. The main content area has a white background with a blue border. It features a "Reset Password" heading, followed by the instruction "Please enter your email address to request a password reset." Below this is a text input field labeled "Email Address". A green button labeled "Reset Password" is positioned below the input field. At the bottom of the white area, there is a link that says "Go back to ? [Log In](#)".

La page de connexion sera initialement présentée à un utilisateur et, une fois l'authentification réussie, l'application accédera à son carnet d'adresses.

Activation de l'authentification Firebase

Accédez à Google Firebase et sélectionnez **Authentification** dans le menu de la console du projet, puis cliquez sur **Get started**.



Firebase permet l'authentification à l'aide de plusieurs fournisseurs de connexion. Nous utiliserons la base de données Firestore stockant l'E-mail/mot de passe (*Email/Password*) et *Google* comme fournisseurs pour cette application.

Cliquez pour activer les fournisseurs de connexion *Email/Password* et *Google*. Pour Google, vous devez fournir une adresse e-mail pour le support du projet. Cette adresse e-mail sera présentée aux utilisateurs lorsqu'ils s'authentifieront auprès de Google comme contact.

Instance Auth

Configurez la fourniture d'authentification dans `src/app/app.config.ts`. Les éléments ajoutés sont affichés ici.

```
1. ...
2. import {getAuth, provideAuth} from "@angular/fire/auth";
3.
4. export const appConfig: ApplicationConfig = {
5.   providers: [
6.     ...
7.     provideAuth() => getAuth()
8.   ]
9. };
```

Routing

Modifiez `/src/app/app-routes.ts` pour spécifier des routes comme suit.

```
1. import { Routes } from '@angular/router';
2. import {AuthGuard} from "@angular/fire/auth-guard";
3.
4. export const routes: Routes = [
5.   { path: "", redirectTo: '/sign-in', pathMatch: 'full' },
6.   { path: 'sign-in', component: SignIn },
7.   { path: 'sign-up', component: SignUp },
8.   { path: 'address-list', component: AddressList, canActivate: [AuthGuard] },
9.   { path: 'password-forgot', component: PasswordForgot }
10. ];
```

Nous avons défini des routes vers un composant Sign Up, un composant Sign In, un composant Password Forgot et l'application principale, le composant Address List. Nous utilisons `AngularFireAuthGuard`, une garde fournie par `AngularFire` pour assurer que seuls les utilisateurs authentifiés puissent accéder au composant Address List.

Générez les composants Sign-In, Sign-Up et Password-Forgot: `ng generate component sign-in`, `ng generate component sign-up`, `ng generate component password-forgot`. Ajoutez des imports des composants générés dans `/src/app/app-routes.ts`.

Mettez à jour le modèle HTML du composant APP (`src/app/app.html`) avec le *router outlet*.

```
1. <app-header></app-header>
2. <div class="container">
3.   <router-outlet></router-outlet>
4. </div>
```

Ajoutez `RouterOutlet` au tableau d'imports du composant App.

Styling CSS

Modifiez le fichier CSS de l'application `src/styles.css` selon le code source sur GitHub.

Service d'authentification

Exécutez `ng generate service authentication/auth-service` pour créer un service pour l'interaction avec les services d'authentification de Firebase.

Modifiez `src/app/authentication/auth-service.ts` tel que suit.

```
1. import { Injectable, inject } from '@angular/core';
2. import { Subscription } from 'rxjs';
3. import {
4.   Auth, User, user,
5.   createUserWithEmailAndPassword,
6.   sendPasswordResetEmail,
7.   signInWithEmailAndPassword, signInWithPopup, signOut,
8.   GoogleAuthProvider, updateProfile
9. } from "@angular/fire/auth";
10.
11. @Injectable({
12.   providedIn: 'root'
13. })
14. export class AuthService {
15.   private afAuth: Auth = inject(Auth);
16.   private loggedInUser: User | null = null;
17.   user$ = user(this.afAuth);
18.   userSubscription: Subscription;
19.
20.   constructor() {
21.     this.userSubscription = this.user$.subscribe((aUser: User | null) => {
22.       this.loggedInUser = aUser;
23.     });
24.   }
25.
26.   get userid(): string {
27.     if (this.loggedInUser) return this.loggedInUser.uid;
28.     return "";
29.   }
30.
31.   signIn(email: string, password: string): Promise<void> {
32.     return signInWithEmailAndPassword(this.afAuth, email, password)
33.       .then((credential) => {
34.         this.loggedInUser = credential.user;
35.       });
36.   }
37.
38.   signUp(email: string, password: string, name: string): Promise<void> {
39.     return createUserWithEmailAndPassword(this.afAuth, email, password)
40.       .then((credential) => {
```

```

41.     const user = credential.user;
42.     this.loggedUser = credential.user; // shouldn't be null...
43.     if (user) {
44.         updateProfile(user, {displayName: name}).then(_ => {});
45.     }
46. });
47. }
48.
49. passwordReset(passwordResetEmail: string): Promise<void> {
50.     return sendPasswordResetEmail(this.afAuth, passwordResetEmail);
51. }
52.
53. googleAuth(): Promise<void> {
54.     const provider = new GoogleAuthProvider();
55.     return signInWithPopup(this.afAuth, provider)
56.         .then((credential) => {
57.             this.loggedUser = credential.user;
58.         });
59. }
60.
61. signOut(): Promise<void> {
62.     return signOut(this.afAuth)
63.         .then(_ => {
64.             this.loggedUser = null;
65.         });
66. }
67. }

```

Firebase fournit une API pour les services d'authentification. Le service Firebase Auth est représenté comme une instance d'`Auth`. Cet objet contient diverses propriétés, y compris l'utilisateur actuellement connecté, le cas échéant. On injecte l'instance à la ligne 15 et on la passe en argument aux différentes fonctions de l'API. Elle est utilisée à la ligne 32 pour la connexion par e-mail/mot de passe, à la ligne 39 pour l'inscription, à la ligne 50 pour la réinitialisation du mot de passe, à la ligne 55 pour la connexion avec Google et à la ligne 60 pour se déconnecter. Les fonctions de l'API Firebase retournent des **Promises** auxquelles on s'abonne pour obtenir les résultats des opérations. Nous définissons la propriété *loggedUser* sur l'utilisateur connecté et retournons l'id de l'utilisateur connecté aux lignes 26-29.

Composant Sign-Up

Modifiez le modèle HTML du composant Sign Up `/src/app/sign-up/sign-up.html` comme suit.

```

1. <div class="displayTable">
2.   <div class="displayTableCell">
3.     <div class="authBlock">
4.       <h3>Sign Up</h3>

```

```

5. <form [formGroup]="signupForm" (ngSubmit)="register()">
6.   <div class="form-group">
7.     <label for="email">Email:</label>
8.     <input type="text" class="form-control" id="email" formControlName="email">
9.     <div [hidden]="email.pristine || email.valid"
10.       class="alert alert-danger">
11.       Well formatted email is required.
12.     </div>
13.   </div>
14.   <div class="form-group">
15.     <label for="name">Name:</label>
16.     <input type="text" class="form-control" id="name" formControlName="name">
17.     <div [hidden]="name.pristine || name.valid"
18.       class="alert alert-danger">
19.       A name must be provided.
20.     </div>
21.   </div>
22.   <div formGroupName="pwGroup">
23.     <div class="form-group">
24.       <label for="password">Password:</label>
25.       <input type="password" class="form-control" id="password" required
26.         formControlName="password">
27.       <div [hidden]="password.pristine || password.valid"
28.         class="alert alert-danger">
29.         A Password is required.
30.       </div>
31.     </div>
32.     <div class="form-group">
33.       <label for="pwconfirm">Confirm Password:</label>
34.       <input type="password" class="form-control" id="pwconfirm" required
35.         formControlName="confirmPassword">
36.       <div [hidden]="confirmPassword.pristine || confirmPassword.valid"
37.         class="alert alert-danger">
38.         A Confirmation Password is required.
39.       </div>
40.     </div>
41.     <div [hidden]="(password.pristine || confirmPassword.pristine) || pwGroup.valid"
42.       class="alert alert-danger">
43.       The passwords do not match.
44.     </div>
45.   </div>
46.   <button type="submit" class="btn btn-success" [disabled]="signupForm.invalid">Register</button>
47. </form>
48. <div class="formGroup">
49.   <span class="or"><span class="orInner">Or</span></span>
50. </div>
51. <!-- Continue with Google -->
52. <div class="formGroup">
53.   <button type="button" class="btn googleBtn" (click)="googleSignIn()">

```

```

54.     Continue with Google
55.     </button>
56.   </div>
57. </div>
58. <div class="redirectToLogin">
59.   <span>Already have an account? <span class="redirect" routerLink="/sign-in">Log
    In</span></span>
60. </div>
61. </div>
62. </div>

```

Nous utilisons un formulaire réactif pour obtenir les données d'inscription et effectuons les validations usuelles.

Modifiez la classe du composant comme suit.

```

1. import {Component, inject} from '@angular/core';
2. import {AbstractControl, ReactiveFormsModule, UntypedFormBuilder, ValidationErrors, Validators} from
   "@angular/forms";
3. import {Router, RouterLink} from "@angular/router";
4. import {AuthService} from "../authentication/auth-service";
5.
6. function passwordMatcher(pwGrp: AbstractControl): ValidationErrors | null {
7.   const passwd = pwGrp.get('password');
8.   const confpasswd = pwGrp.get('confirmPassword');
9.   return passwd!.value === confpasswd!.value ? null : {mismatch: true};
10. }
11.
12. @Component({
13.   selector: 'app-sign-up',
14.   standalone: true,
15.   imports: [
16.     ReactiveFormsModule,
17.     RouterLink
18.   ],
19.   templateUrl: './sign-up.html',
20.   styleUrls: ['./sign-up.css']
21. })
22. export class SignUp {
23.   private builder: UntypedFormBuilder = inject(UntypedFormBuilder);
24.   private authService: AuthService = inject(AuthService);
25.   private router: Router = inject(Router);
26.   signupForm = this.builder.group({
27.     email: ["", [Validators.required, Validators.email]],
28.     name: ["", Validators.required],
29.     pwGroup: this.builder.group({
30.       password: ["", Validators.required],
31.       confirmPassword: ["", Validators.required]
32.     }, { validators: [passwordMatcher] })
33.   });

```

```

34.
35. get email(): AbstractControl {return <AbstractControl>this.signupForm.get('email'); }
36. get name(): AbstractControl {return <AbstractControl>this.signupForm.get('name'); }
37. get password(): AbstractControl {return
    <AbstractControl>this.signupForm.get('pwGroup')!.get('password'); }
38. get confirmPassword(): AbstractControl {return
    <AbstractControl>this.signupForm.get('pwGroup')!.get('confirmPassword'); }
39. get pwGroup(): AbstractControl {return <AbstractControl>this.signupForm.get('pwGroup'); }
40.
41. register(): void {
42.   this.authService.signUp(this.email.value, this.password.value, this.name.value)
43.     .then(() => {
44.       this.router.navigate(['address-list']).then(() => {});
45.     })
46.     .catch((error) => {
47.       window.alert(error.message);
48.     });
49. }
50.
51. googleSignIn(): void {
52.   this.authService.googleAuth()
53.     .then(() => {
54.       this.router.navigate(['address-list']).then(() => {});
55.     })
56.     .catch((error) => {
57.       window.alert(error.message);
58.     });
59. }
60. }

```

Le service d'authentification est injecté (ligne 24) et utilisé dans le gestionnaire pour l'inscription (lignes 41-49). Lorsqu'un utilisateur choisit de continuer avec Google, nous appelons une fonction pour effectuer la connexion avec Google (lignes 51-59). Une fois la connexion réussie, l'application navigue à la page de la liste d'adresses.

Composant Sign-In

Modifiez le modèle HTML du composant de connexion `/src/app/sign-in/sign-in.html` comme suit.

```

1. <div class="displayTable">
2.   <div class="displayTableCell">
3.     <div class="authBlock pt-5">
4.       <h3>Sign In</h3>
5.       <div class="formGroup">
6.         <input type="text" class="formControl" placeholder="Email" #email required>
7.       </div>
8.       <div class="formGroup">
9.         <input type="password" class="formControl" placeholder="Password" #userPassword required>
10.      </div>

```

```

11. <div class="formGroup">
12.   <input type="button" class="btn btn-outline-primary" value="Log in"
13.     (click)="signIn(email.value, userPassword.value)">
14. </div>
15. <div class="formGroup">
16.   <span class="or"><span class="orInner">Or</span></span>
17. </div>
18. <div class="formGroup">
19.   <button type="button" class="btn btn-outline-primary" (click)="googleSignIn()">
20.     Log in with Google
21.   </button>
22. </div>
23. <div class="forgotPassword">
24.   <span routerLink="/password-forgot">Forgot Password?</span>
25. </div>
26. </div>
27. <div class="redirectToLogin">
28.   <span>
29.     Don't have an account?
30.     <span class="redirect" routerLink="/sign-up"> Sign Up</span>
31.   </span>
32. </div>
33. </div>
34. </div>

```

Modifiez la classe du composant comme suit.

```

1. import {Component, inject} from '@angular/core';
2. import {Router, RouterLink} from "@angular/router";
3. import {AuthService} from "../authentication/auth-service";
4.
5. @Component({
6.   selector: 'app-sign-in',
7.   standalone: true,
8.   imports: [
9.     RouterLink
10.  ],
11.   templateUrl: './sign-in.html',
12.   styleUrls: ['./sign-in.css']
13. })
14. export class SignIn {
15.   private authService: AuthService = inject(AuthService);
16.   private router: Router = inject(Router);
17.
18.   signIn(email: string, password: string): void {
19.     this.authService.signIn(email, password)
20.       .then(() => {
21.         this.router.navigate(['address-list']).then(() => {});

```

```

22.   })
23.   .catch((error) => {
24.     window.alert(error.message);
25.   });
26. }
27.
28. googleSignIn(): void {
29.   this.authService.googleAuth()
30.   .then(() => {
31.     this.router.navigate(['address-list']).then(() => {});
32.   })
33.   .catch((error) => {
34.     window.alert(error.message);
35.   });
36. }
37. }

```

Nous injectons le service d'authentification à la ligne 15 et l'utilisons dans les gestionnaires pour la connexion (lignes 18-26) et la connexion avec Google (lignes 28-36).

Composant Password-Forgot

Modifiez le modèle HTML du composant de réinitialisation de mot de passe /src/app/password-forgot/password-forgot.html comme suit.

```

1. <div class="displayTable">
2.   <div class="displayTableCell">
3.     <div class="authBlock">
4.       <h3>Reset Password</h3>
5.       <p class="text-center">Please enter your email address to request a password reset.</p>
6.       <div class="formGroup">
7.         <input type="email" class="formControl"
8.           placeholder="Email Address" #passwordResetEmail required>
9.       </div>
10.      <div class="formGroup">
11.        <input type="submit" class="btn btnPrimary" value="Reset Password"
12.          (click)="passwordReset(passwordResetEmail.value)">
13.      </div>
14.    </div>
15.    <div class="redirectToLogin">
16.      <span>Go back to ? <span class="redirect" routerLink="/sign-in">Log In</span></span>
17.    </div>
18.  </div>
19. </div>

```

Modifiez la classe du composant comme suit.

```

1. import {Component, inject} from '@angular/core';
2. import {RouterLink} from "@angular/router";
3. import {AuthService} from "../authentication/auth-service";
4.

```

```

5. @Component({
6.   selector: 'app-password-forgot',
7.   standalone: true,
8.   imports: [
9.     RouterLink
10.  ],
11.   templateUrl: './password-forgot.html',
12.   styleUrls: ['./password-forgot.css']
13. })
14. export class PasswordForgot {
15.   private authService: AuthService = inject(AuthService);
16.
17.   passwordReset(email: string): void {
18.     this.authService.passwordReset(email)
19.       .then(() => {
20.         window.alert('Password reset email sent, check your inbox.');
```

Composant Header

Nous mettons à jour le composant d'en-tête pour afficher l'utilisateur connecté et présenter un bouton de déconnexion. Modifiez `src/app/header/header.html` comme suit.

```

1. <nav class="navbar navbar-expand-lg navbar-fixed-top shadow">
2.   <div class="container-fluid">
3.     <div class="navbar-header">
4.       <a href="#" class="navbar-brand">Address Book</a>
5.     </div>
6.     @if (user | async; as user) {
7.       <div>
8.         <ul class="navbar-nav mr-auto">
9.           <li class="nav-item active">
10.            <span class="navbar-text">{{user.displayName}}</span>
11.          </li>
12.          <li class="nav-item active">
13.            <a class="nav-link" (click)="signOut()">Sign Out </a>
14.          </li>
15.        </ul>
16.      </div>
17.    }
18.  </div>
19. </nav>
```

Modifiez `src/app/header/header.ts` comme suit.

```

1. import {Component, inject} from '@angular/core';
2. import {AsyncPipe, NgIf} from '@angular/common';
```

```

3. import {User} from "@angular/fire/auth";
4. import {AuthService} from "../authentication/auth-service";
5. import {Router} from "@angular/router";
6. import {Observable} from "rxjs";
7.
8. @Component({
9.   selector: 'app-header',
10.  standalone: true,
11.  imports: [
12.    NgIf,
13.    AsyncPipe
14.  ],
15.  templateUrl: './header.html',
16.  styleUrls: ['./header.css']
17. })
18. export class Header {
19.   private authService: AuthService = inject(AuthService);
20.   private router: Router = inject(Router);
21.
22.   get user(): Observable<User | null> {
23.     return this.authService.user$;
24.   }
25.
26.   signOut(): void {
27.     this.authService.signOut().then(_) => {
28.       this.router.navigate(['sign-in']).then(() => {});
29.     });
30.   }
31. }

```

Service Firestore

Afin d'associer chacun des utilisateurs de l'application à sa propre liste d'adresses, nous allons subdiviser la collection de carnets d'adresses (**abooks**) en documents dédiés chacun à un utilisateur, et identifiés avec l'identifiant de cet utilisateur. Modifiez `src/app/address-list/firestore/address-db-service.ts` comme suit.

```

1. import { Injectable, inject } from '@angular/core';
2. import {
3.   Firestore,
4.   collection,
5.   collectionData,
6.   addDoc,
7.   doc,
8.   setDoc,
9.   deleteDoc
10. } from '@angular/fire/firestore';
11. import { Observable } from "rxjs";

```

```

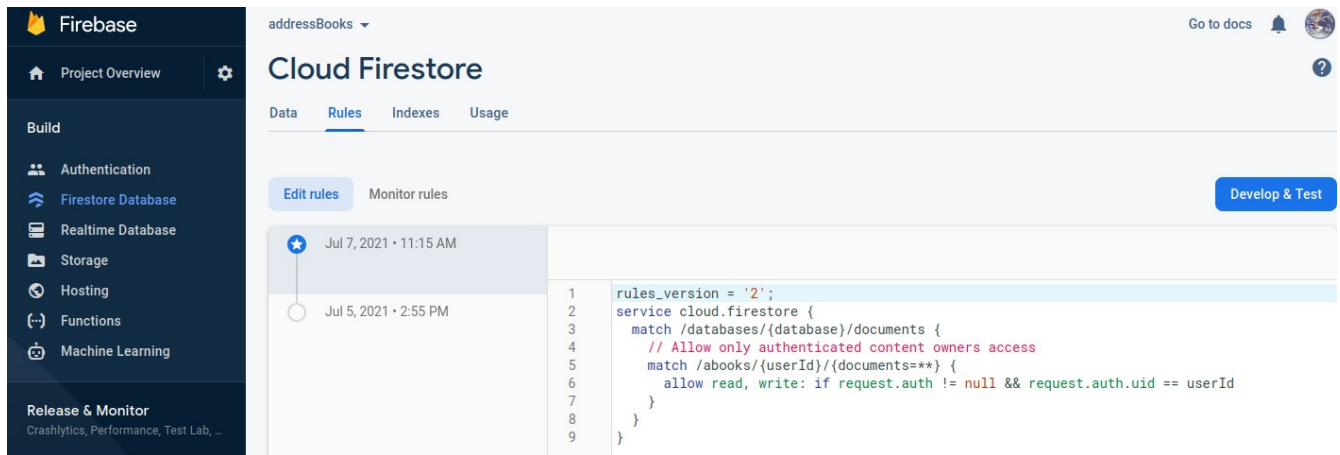
12. import {AddressEntry} from "../address-entry";
13. import {AuthService} from "../authentication/auth.service";
14.
15. @Injectable({
16.   providedIn: 'root'
17. })
18. export class AddressDbService {
19.   private firestore: Firestore = inject(Firestore);
20.   private authService: AuthService = inject(AuthService);
21.
22.   getAddresses(): Observable<AddressEntry[]> {
23.     const addresses = collection(this.firestore, 'abooks', this.authService.userid, 'addresses');
24.     return collectionData(addresses, {idField: 'id'}) as Observable<AddressEntry[]>;
25.   }
26.
27.   createAddress(address: AddressEntry) {
28.     const addresses = collection(this.firestore, 'abooks', this.authService.userid, 'addresses');
29.     delete address.id;
30.     // @ts-ignore
31.     return addDoc(addresses, address);
32.   }
33.
34.   updateAddress(address: AddressEntry) {
35.     const addressId = address.id;
36.     delete address.id;
37.     const addresses = collection(this.firestore, 'abooks', this.authService.userid, 'addresses');
38.     const addressRef = doc(addresses, addressId!);
39.     // @ts-ignore
40.     return setDoc(addressRef, address);
41.   }
42.
43.   deleteAddress(addressId: string): Promise<void> {
44.     const addresses = collection(this.firestore, 'abooks', this.authService.userid, 'addresses');
45.     const addressRef = doc(addresses, addressId);
46.     return deleteDoc(addressRef);
47.   }
48. }

```

Nous injectons le service d'authentification à la ligne 20 et utilisons l'identifiant de l'utilisateur authentifié pour identifier un document aux lignes 23, 28, 37, et 44 . Cela fournit un document spécifique à chaque utilisateur dans le magasin de données.

Sécurisation de Firestore

Google Firebase nous permet d'ajouter des règles d'accès aux bases de données Cloud Firestore afin d'éviter tout accès non autorisé aux données d'applications.



Dans l'exemple ci-dessus, nous avons ajouté une règle par laquelle une requête de lecture ou d'écriture d'un document n'est acceptée que d'un utilisateur authentifié dont l'ID usager correspond à l'identifiant du document.

Vous pouvez en savoir plus sur les règles de sécurité de Firestore Cloud à <https://firebase.google.com/docs/firestore/security/get-started>.

Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Lorsque vous accepterez le devoir pour ce lab sur Github, votre référentiel sera initialisé par du code d'une application de démarrage. Étendre l'application de démarrage fournie pour :

1. persister les informations des employés entrées dans un Cloud Datastore Firestore,
2. permettre la récupération à partir du Firestore Cloud Datastore et l'affichage de toutes les informations des employés enregistrées dans un tableau lorsque le bouton « Employee List » est cliqué.

Vous devez spécifier les valeurs d'environnement appropriées de façon à ce que l'accès à votre base de données Firestore soit possible.